

Universidad de La Habana
Facultad de Matemática y Computación



Sistema web para el autorreporte de Eventos Supuestamente Atribuibles a la Vacunación o Inmunización (ESAVI) en el Instituto Finlay de Vacunas (IFV)

Autor:

Jabel Resendiz Aguirre

Tutores:

MsC. Joanna Campbell Amos

MsC. Aracelys García Armenteros

Trabajo de Diploma
presentado en opción al título de
Licenciado en Ciencia de la Computación

17 de junio de 2026

github.com/JabelResendiz/thesis

Resumen

La implementación del sistema web para la gestión de ESAVI se fundamenta en una arquitectura modular basada en los principios de Clean Architecture, utilizando .NET 8 para el backend y React con TypeScript para el frontend. La solución integra mecanismos avanzados de seguridad, incluyendo autenticación mediante JWT con rotación de refresh tokens almacenados en cookies HttpOnly, limitación de velocidad (rate limiting) y protección contra ataques automatizados mediante Friendly CAPTCHA. Para optimizar la experiencia del usuario y la fiabilidad, se implementó una comunicación asíncrona empleando RabbitMQ y MassTransit, facilitando notificaciones automáticas vía correo electrónico y WhatsApp. El sistema garantiza la integridad y trazabilidad de los datos de farmacovigilancia mediante el uso del patrón Unit of Work y un registro de auditoría automatizado sobre una base de datos MySQL.

Abstract

The implementation of the ESAVI management web system is based on a modular architecture following Clean Architecture principles. The backend was developed using .NET 8 and C#, while the frontend was built as a Single Page Application (SPA) with React and TypeScript. The system features a robust security framework, including JWT authentication with Refresh Token rotation (HttpOnly cookies), IP-based rate limiting, and Friendly CAPTCHA protection for public endpoints. To ensure high availability and responsiveness, the solution incorporates asynchronous messaging via RabbitMQ and MassTransit for automated email and WhatsApp notifications. Data integrity and traceability are managed through Entity Framework Core, the Unit of Work pattern, and an automated Audit Log system.

Capítulo 1

Validación de la aplicación

El Capítulo ?? describió los detalles de implementación del sistema: la arquitectura, las tecnologías empleadas y las decisiones de diseño. Sin embargo, la implementación por sí sola no garantiza que el sistema cumpla con los requerimientos definidos ni que ofrezca una experiencia adecuada a sus usuarios. Este capítulo somete el producto a un proceso de validación que abarca cuatro dimensiones complementarias: corrección funcional, rendimiento bajo carga, calidad del despliegue web y usabilidad percibida.

1.1. Propósito de la Validación

La validación del sistema persigue demostrar cuatro afirmaciones concretas:

1. **Los flujos principales operan correctamente.** El registro de reportes, la asignación a revisores, la gestión de revisiones y las notificaciones automáticas funcionan conforme a los requerimientos funcionales definidos en el Capítulo ??.
2. **El sistema soporta la concurrencia esperada.** Los endpoints críticos —creación y búsqueda de reportes— responden bajo carga concurrente sin superar los umbrales de tiempo definidos, incluso con volúmenes de datos acumulados.
3. **La calidad web cumple con los estándares.** El frontend desplegado satisface los criterios de rendimiento, accesibilidad, mejores prácticas y SEO evaluados mediante Lighthouse.
4. **Los usuarios finales consideran la interfaz útil y fácil de usar.** La evaluación mediante el cuestionario SUS (System Usability Scale) debe arrojar una puntuación superior a 68, umbral que define la usabilidad aceptable, y preferiblemente superior a 80.3, categoría “excelente” según Bangor et al. [3].

1.2. Enfoques de Validación

La validación abordó las cuatro afirmaciones mediante los siguientes enfoques complementarios:

- **Pruebas funcionales:** ejecución de los flujos principales del sistema, verificando que cada paso produce el resultado esperado según los requerimientos. Cada caso de prueba detalla las características del entorno de ejecución.
- **Pruebas de rendimiento:** evaluación de la capacidad del sistema bajo carga concurrente mediante k6, con el backend y el cliente en máquinas independientes conectadas por red hotspot. La Sección ?? presenta las especificaciones del hardware.
- **Validación de calidad web:** auditoría del frontend desplegado en Vercel mediante PageSpeed Insights (Lighthouse), evaluando rendimiento, accesibilidad, mejores prácticas y SEO.
- **Pruebas de usabilidad:** recorrido cognitivo con 7 participantes —5 ciudadanos y 2 administradores— representativos de los perfiles de uso del sistema, complementado con el cuestionario SUS.

1.3. Pruebas funcionales

El objetivo de estas pruebas fue validar que las funcionalidades críticas del sistema ESAVI cumplen con los requisitos de diseño y garantizan el flujo de información para la farmacovigilancia. La estrategia empleada fue de caja negra, simulando operaciones de usuarios normales desde el navegador con datos sintéticos (*mock*) para verificar la persistencia y la respuesta del sistema ante entradas representativas.

La Tabla 1.1 resume el estado de cumplimiento de las funciones principales: todos los casos de prueba seleccionados resultaron exitosos, confirmando la operatividad de la plataforma.

1.3.1. Evidencias de ejecución

En esta sección se presentan capturas de pantalla que evidencian la ejecución correcta de los principales flujos del sistema, validando la interacción entre el usuario y la interfaz, así como la respuesta del servidor en tiempo real.

Cada flujo funcional se documenta de manera secuencial, permitiendo demostrar de forma estructurada el comportamiento integral del sistema de extremo a extremo.

Tabla 1.1: Resumen de casos de prueba funcionales

ID	Caso de prueba	Datos de entrada	Resultado esperado	Resultado obtenido	¿Pasó?
CP01	Inicio de sesión	Credenciales válidas	Acceso al sistema según el rol del usuario	Acceso concedido correctamente	✓
CP02	Registro de reporte ESAVI	Datos completos del reporte	Reporte almacenado y documento PDF generado	Operación realizada correctamente	✓
CP03	Consulta por número de notificación	Número de notificación existente	Visualización del estado y detalles del reporte	Información mostrada correctamente	✓
CP04	Gestión de usuarios	Datos de un nuevo médico o jefe de sección	Usuario registrado y disponible para administración	Usuario creado correctamente	✓
CP05	Asignación de reporte a médico	Reporte pendiente y médico disponible	Asignación creada y visible para el médico	Asignación realizada correctamente	✓
CP06	Resolución de reportes duplicados	Reportes identificados como posibles duplicados	Actualización del estado según la decisión tomada	Proceso ejecutado correctamente	✓
CP07	Visualización del dashboard administrativo	Solicitud de acceso al panel de indicadores	Presentación de estadísticas e indicadores	Dashboard mostrado correctamente	✓
CP08	Filtrado de reportes	Parámetros de búsqueda y filtrado	Listado de reportes coincidentes	Resultados obtenidos correctamente	✓
CP09	Revisión médica del reporte	Reporte asignado al médico	Actualización del estado y registro de la revisión	Revisión completada correctamente	✓
CP10	Inicio de sesión con credenciales inválidas	Usuario o contraseña incorrectos	Rechazo del acceso y mensaje de error	Validación ejecutada correctamente	✓
CP11	Registro de reporte con datos obligatorios ausentes	Formulario incompleto	Notificación de errores de validación	Validación ejecutada correctamente	✓
CP12	Consulta de número de notificación inexistente	Número no registrado	Notificación de reporte no encontrado	Respuesta emitida correctamente	✓

Flujo completo del manejo de reportes ESAVI

Este flujo describe el ciclo completo de gestión de un reporte ESAVI, desde su creación por el usuario hasta su evaluación final por el médico evaluador y la consulta del estado actualizado del caso, como se muestra en la Figura 1.2.

Gestión de acceso y registro de médico evaluador

Este flujo, ilustrado en la Figura 1.2, abarca el registro de un médico evaluador por parte del jefe municipal (Figura 1.2a), el envío de una invitación por correo electrónico (Figura 1.2b), el establecimiento de contraseña (Figura 1.2c) y el posterior inicio de sesión en el sistema (Figura 1.2d), garantizando un acceso controlado al módulo de revisión de reportes.

(a) Creación de reporte ESAVI

(b) Confirmación de registro del reporte

(c) Asignación del reporte por el jefe de vacunación municipal a un médico evaluador

(d) Visualización del reporte por el médico evaluador asignado

(e) Actualización del estado del reporte tras la evaluación médica

Figura 1.1: Flujo del sistema ESAVI para la creación, asignación y evaluación de reportes de eventos adversos posteriores a la vacunación

Gestionar Médicos Revisores

Total de médicos revisores en el municipio: 16

Agregar Nuevo Médico Revisor

Nombre de Usuario * Juan Perez	Email * [redacted]00@gmail.com
Teléfono * 55 [redacted]	
Institución * Consultorio #23	Especialidad * Medicina General
Número de Registro Profesional * MED-19821-7890	Número de Identidad * 01012345678

[Registrar Médico](#) [Cancelar](#)

(a) Registro de médico evaluador por el jefe municipal



Invitación a la Plataforma de Farmacovigilancia

Se ha creado una cuenta para usted dentro del sistema de Farmacovigilancia del Instituto Finlay de Vacunas.

Para activar su cuenta y establecer sus credenciales de acceso, debe completar el proceso de verificación.

Durante la activación se le solicitará su **número de registro médico** como medida de validación de identidad.

[Activar cuenta](#)

Este enlace expirará en **24 horas** por motivos de seguridad. Si usted no esperaba esta invitación, puede ignorar este correo.

Atentamente,
Sistema de Farmacovigilancia — Instituto Finlay de Vacunas

Este correo fue enviado a [redacted]00@gmail.com

Email sent via [EmailJS.com](#)

(b) Correo de invitación con enlace para establecer contraseña

Completa tu registro

Ingresa tu número de registro profesional y tu nueva contraseña para activar tu cuenta.

Email de activación
[redacted]00@gmail.com

Número de registro profesional
MED-19821-7890

Nueva contraseña
.....

Confirmar contraseña
.....

[Completar registro](#)

[Volver a iniciar sesión](#)

(c) Página de establecimiento de contraseña

IFV INSTITUTO FINLAY DE VACUNAS

[Inicio](#) [Reportes Asignados](#) [Información](#)

JuanPerez (Profesional Evaluador) [Salir](#)

Sistema de Farmacovigilancia

Instituto Finlay de Vacunas

Plataforma de monitoreo y reporte de eventos adversos asociados a vacunas. Contribuya a la seguridad y eficacia de las inmunizaciones en Cuba.

[Ver Reportes Asignados](#)

(d) Pantalla de inicio tras autenticación exitosa

Figura 1.2: Flujo de registro y activación de un médico evaluador en el sistema ESAVI.

1.3.2. Análisis de resultados

Los 12 casos de prueba funcionales fueron ejecutados con éxito, sin desviaciones entre los resultados esperados y los obtenidos. Los flujos principales —creación, asignación, revisión y consulta de reportes— operaron de extremo a extremo conforme a los requerimientos. El registro y activación de médicos evaluadores confirmó el control de acceso al sistema, y los casos negativos validaron que el sistema rechaza correctamente entradas inválidas. En conjunto, las pruebas funcionales demuestran que las funcionalidades críticas del sistema operan correctamente.

1.4. Pruebas de rendimiento

1.4.1. Entorno de Pruebas

Las pruebas de rendimiento se ejecutaron con k6, herramienta de código abierto para pruebas de carga. El backend se desplegó en una máquina con sistema operativo Linux Ubuntu 22.04, procesador Intel Core i5 de 4 núcleos a 2.5 GHz, 8 GB de RAM y almacenamiento SSD de 256 GB. k6 se ejecutó en una máquina cliente independiente con Windows 11, procesador Intel Core i7 de 8 núcleos a 3.0 GHz y 16 GB de RAM, conectadas mediante red hotspot a 2.4 GHz. Esta separación garantiza que las mediciones no se vean afectadas por competencia de recursos entre el cliente de pruebas y el servidor.

Se emplearon datos simulados que representan solicitudes realistas con una carga útil equivalente a la del sistema en producción. Las métricas analizadas fueron: latencia promedio y percentil 95 (`http_req_duration`), throughput (`http_reqs`) y tasa de solicitudes exitosas (`status 201`). Se definió como umbral de tiempo aceptable 3000 ms y como criterio de fallo funcional cualquier código de estado distinto de 2XX.

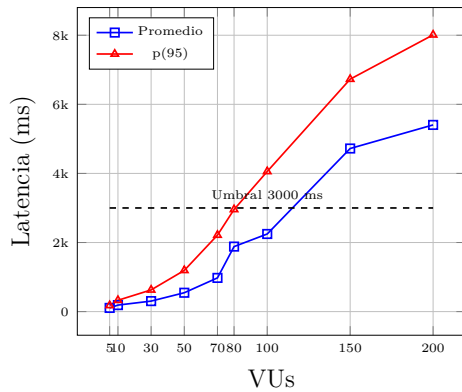
1.4.2. Estrategia de pruebas

Se aplicaron dos enfoques complementarios. Por un lado, pruebas de estrés sobre las operaciones públicas de creación y búsqueda de reportes, por ser los que recibirán el mayor volumen de tráfico al estar expuestos a todos los ciudadanos. El resto de operaciones —consulta de pendientes, asignación a médicos, evaluación y dashboard— están protegidas por autenticación y su concurrencia está acotada por el número de usuarios registrados en cada rol, que no varía en períodos cortos de tiempo. Por otro lado, un escenario combinado que integra todos los perfiles de uso con pausas realistas, para validar el comportamiento del sistema en condiciones normales de operación.

Creación de Reporte de ESAVI

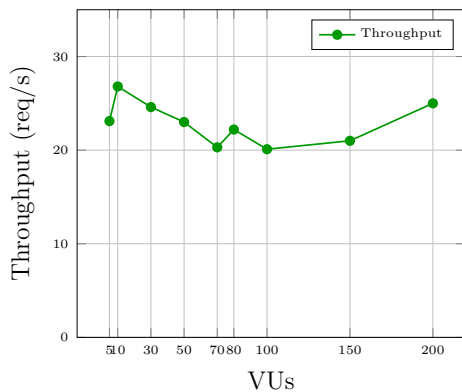
Para evaluar el rendimiento de la creación de reportes se diseñó una prueba de estrés incremental: los usuarios virtuales envían solicitudes de forma ininterrumpida, sin pausas, con el objetivo de llevar al sistema al límite e identificar el punto en que la latencia supera los 3000 ms o aparecen fallos. Cada solicitud contiene un reporte completo con 13 vacunas y 10 lotes por vacuna. La prueba se configuró con etapas de rampa (1 min de subida, 3 min estable, 1 min de bajada) y se ejecutó de forma independiente para 5, 10, 30, 50, 70, 80, 100, 150 y 200 VUs. Los detalles de configuración, generación de datos y código k6 se presentan en el Anexo A.1.

Las Figuras 1.3, 1.4 y 1.5 muestran la evolución de las métricas conforme aumenta la carga concurrente.



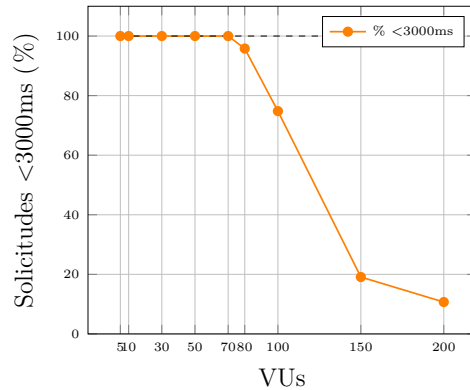
Los resultados muestran tres zonas de comportamiento. Hasta 70 VUs, la latencia se mantiene por debajo de 1000 ms con el 100% de las solicitudes respondiendo en menos de 3000 ms. A partir de 80 VUs aparece el primer punto de degradación: la latencia asciende a 1882 ms. Con 100 VUs alcanza 2246 ms, y con 200 VUs supera los 5400 ms.

Figura 1.3: Evolución de la latencia según usuarios concurrentes.



El throughput se mantuvo estable entre 20 y 26 req/s en todos los niveles de carga. Esto indica que el sistema alcanzó su capacidad máxima de procesamiento sin degradar la tasa de solicitudes exitosas: los usuarios adicionales no aumentan el rendimiento, sino que se traducen en mayor latencia.

Figura 1.4: Evolución del throughput según usuarios concurrentes.



Hasta 70 VUs, el 100% de las solicitudes responde en menos de 3000 ms. A partir de 80 VUs este porcentaje comienza a caer: 95.8% con 80 VUs, 74.8% con 100 VUs, y se desploma al 19.1% y 10.7% con 150 y 200 VUs respectivamente. En ningún momento se registraron errores funcionales (100% status 201), lo que confirma que la degradación es atribuible a las limitaciones del entorno de pruebas y no a deficiencias del código.

Figura 1.5: Porcentaje de solicitudes por debajo del umbral de 3000 ms.

La prueba de carga sobre el endpoint de creación de reportes permite concluir que el sistema ofrece un rendimiento sólido bajo concurrencia moderada, con una zona de operación óptima donde la experiencia de usuario se mantiene fluida y sin degradación. La prueba simula un escenario más exigente que el uso real —cada usuario virtual envía solicitudes de forma ininterrumpida—, por lo que los resultados representan una cota inferior conservadora de la capacidad real del sistema.

Al superar el umbral de operación óptima, la latencia comienza a degradarse progresivamente, aunque sin comprometer la funcionalidad: en ningún nivel de carga se registraron errores, lo que confirma que las limitaciones observadas son atribuibles al entorno de pruebas y no a deficiencias del código desarrollado.

Búsqueda de Reporte por Número de Notificación

Para evaluar el rendimiento de la búsqueda por notificación se diseñó una prueba de estrés incremental variando dos factores: el tamaño de la base de datos y la concurrencia. Al igual que en la creación, los usuarios virtuales envían solicitudes de forma ininterrumpida, sin pausas, para identificar el punto de saturación. Se pobló la base de datos con 1,000, 4,000, 10,000, 50,000 y 100,000 reportes, y para cada nivel se ejecutaron pruebas independientes con 5, 10, 30, 50, 70, 100, 150 y 200 VUs, usando la misma configuración de rampa (1 min subida, 3 min estable, 1 min bajada). Cada solicitud busca un número de notificación aleatorio extraído de un archivo CSV con los identificadores reales de los reportes almacenados. Los detalles de configuración, generación de datos y código k6 se presentan en el Anexo A.2.

Las Figuras 1.6 y 1.7 muestran la evolución de las métricas conforme aumenta la carga concurrente para cada tamaño de base de datos.

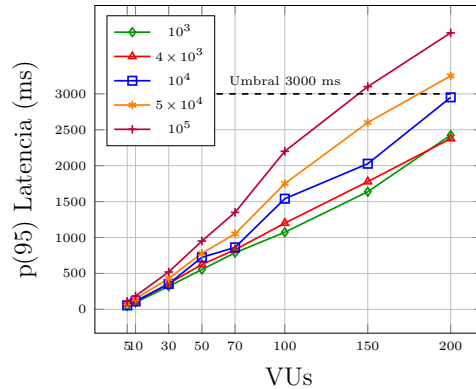


Figura 1.6: Evolución del p(95) de latencia según usuarios concurrentes para 1,000, 4,000, 10,000, 50,000 y 100,000 reportes.

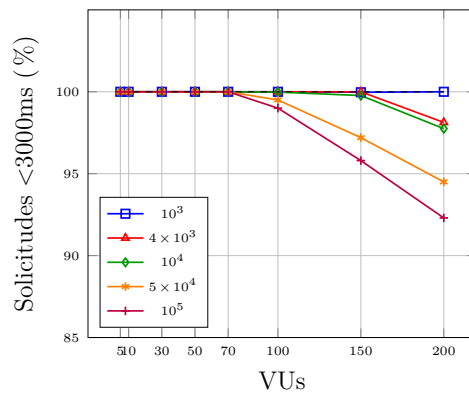


Figura 1.7: Porcentaje de solicitudes por debajo del umbral de 3000 ms según usuarios concurrentes y tamaño de base de datos.

Los resultados confirman que la búsqueda por número de notificación es robusta tanto frente a la concurrencia como al crecimiento del volumen de datos. En todos los niveles evaluados —hasta 200 VUs y 10^5 reportes— el sistema mantuvo una tasa de éxito del 100% (status 202), sin registrar errores funcionales.

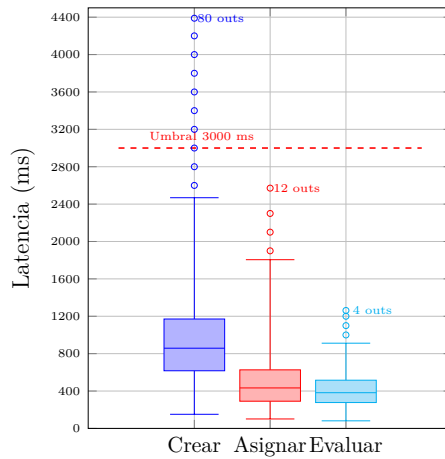
Incluso en las condiciones más exigentes, más del 90% de las solicitudes respondió por debajo del umbral de 3000 ms, lo que evidencia que la búsqueda está fortalecida para operar bajo alta concurrencia con grandes volúmenes de reportes acumulados. La degradación observada es progresiva y predecible, sin caídas abruptas, lo que permite anticipar el comportamiento del sistema ante escenarios de crecimiento futuro.

Hasta 70 VUs, todas las curvas se mantienen agrupadas por debajo de 1000 ms, sin diferencias significativas entre tamaños de base de datos. A partir de 100 VUs las curvas comienzan a separarse: 10^3 , 4×10^3 y 10^4 reportes se mantienen por debajo del umbral de 3000 ms incluso con 200 VUs, mientras que 5×10^4 lo supera con 200 VUs (3250 ms) y 10^5 lo hace desde los 150 VUs (3100 ms). Esto evidencia que el volumen de datos impacta progresivamente en la latencia bajo alta concurrencia.

Con 10^3 , 4×10^3 y 10^4 reportes, más del 97% de las solicitudes responde por debajo de 3000 ms incluso con 200 VUs. Con 5×10^4 y 10^5 reportes, el porcentaje desciende al 94.5% y 92.3% respectivamente con 200 VUs, manteniéndose siempre por encima del 90%. En ningún nivel se registraron errores funcionales (100% status 202).

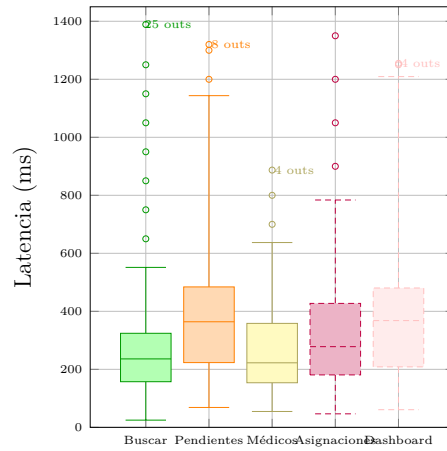
Escenario Combinado

Para validar el comportamiento del sistema bajo una carga de trabajo realista con múltiples operaciones conviviendo, se diseñó un escenario con tres perfiles de tráfico simultáneos: tráfico público (75 VUs, alternando 65% creación de reportes y 35% búsqueda por notificación, con pausas de 1 a 3 segundos), tráfico de jefes municipales (10 VUs, consultando el dashboard y asignando reportes a médicos evaluadores, con pausas de 2 a 6 segundos) y tráfico de médicos (15 VUs, revisando su bandeja de asignaciones y evaluando reportes, con pausas de 1 a 4 segundos). Las métricas se aislaron mediante tags para obtener resultados independientes por cada tipo de operación. Los detalles del script se presentan en el Anexo A.3.



Las operaciones de escritura presentan la mayor latencia del sistema. Crear reporte concentra la mayor dispersión, con una mediana de 858 ms y 80 valores atípicos que superan los 2468 ms, varios por encima de los 3000 ms. Asignar y Evaluar muestran cajas más compactas, con medianas de 433 ms y 382 ms respectivamente, y pocos valores atípicos. Todas las operaciones mantuvieron el 100% de solicitudes exitosas.

Figura 1.8: Distribución de latencias en operaciones de escritura: crear reporte, asignar a médico y evaluar reporte.



Las operaciones de lectura son significativamente más rápidas y compactas que las de escritura. Buscar reporte por notificación es la consulta más eficiente, con una mediana de 236 ms. El resto de las lecturas presentan medianas entre 222 ms y 368 ms, con pocos valores atípicos. Todos los p(95) se mantienen por debajo de los 1400 ms, muy lejos del umbral de 3000 ms, y el 100% de las solicitudes fueron exitosas.

Figura 1.9: Distribución de latencias en operaciones de lectura: búsqueda de reportes, consulta de pendientes, médicos, asignaciones y dashboard.

Tabla 1.2: Métricas globales del escenario combinado.

Métrica	Valor
Solicitudes totales	2,382
Throughput global	36.8 req/s
Latencia promedio global	646.2 ms
Mediana global	471.8 ms
p(95) global	1623.4 ms
Tasa de éxito	99.5%

La Tabla 1.2 resume el desempeño general del sistema en el escenario combinado: 2,382 solicitudes procesadas en un minuto con 100 VUs concurrentes, un throughput de 36.8 req/s y una tasa de éxito del 99.5%.

Tabla 1.3: Métricas detalladas del escenario combinado.

Métrica	Crear	Buscar	Pendientes	Médicos	Asignar	Asignaciones	Evaluar	Dashboard
Solicitudes	1078	581	84	84	84	365	68	38
Promedio	1021.37	268.04	423.15	275.19	603.64	348.86	433.61	428.00
Mediana	858.28	235.99	364.01	222.53	433.23	278.22	382.17	368.06
Q1	616.87	157.37	223.25	153.75	291.09	180.81	276.81	208.54
Q3	1170.59	324.28	484.20	358.44	626.59	427.35	515.97	480.45
p95	2468.39	551.48	1143.59	637.15	1805.32	783.83	911.90	1209.34
Mínimo	151.15	24.89	68.61	54.73	101.22	46.62	81.17	60.88
Máximo	4389.06	1389.79	1320.05	887.75	2571.61	1928.12	1262.24	1255.79
Outliers	80	25	8	4	12	18	4	4

La Tabla 1.3 desglosa las métricas por operación. Las lecturas son un 73.8% más rápidas que las escrituras, y la creación de reportes es la operación más costosa, con una mediana 3.8 veces superior a la búsqueda. Los valores atípicos se concentran en las escrituras, como es esperable por su mayor carga de procesamiento. En conjunto, el sistema demuestra capacidad para operar bajo carga concurrente con múltiples perfiles de usuario, manteniendo latencias controladas y sin interferencia entre operaciones.

1.4.3. Comparación entre pruebas de estrés y escenario realista

Las pruebas individuales de creación y búsqueda se ejecutaron sin pausas entre solicitudes, simulando un escenario de estrés sostenido donde cada usuario virtual envía peticiones de forma ininterrumpida. El escenario combinado, en cambio, incorporó pausas aleatorias de 1 a 6 segundos para reflejar el ritmo real de interacción humana. Esta diferencia metodológica explica las variaciones observadas en los resultados: el throughput global del escenario combinado (36.8 req/s) supera al de las pruebas individuales (20-26 req/s), y las latencias a cargas equivalentes son menores cuando las solicitudes se espacian. Ambos enfoques son complementarios: las pruebas sin pausas establecen el límite de estrés que el sistema tolera sin fallar, mientras que el escenario combinado confirma que, en condiciones reales de operación, el rendimiento es superior y la experiencia del usuario se mantiene fluida.

1.5. Validación del Entorno de Producción y Calidad Web

Esta validación verifica la calidad del sistema una vez desplegado en un entorno real, considerando la experiencia del usuario final bajo distintas condiciones de conectividad. Para ello se empleó PageSpeed Insights, herramienta de Google que ejecuta el motor de auditoría Lighthouse y evalúa la calidad web con base en las métricas definidas en Web Vitals [1]. El sistema, desplegado en Vercel, se auditó simulando dos escenarios: un Moto G Power con red 4G lenta (móvil) y una estación de trabajo con conexión de alta velocidad (escritorio).

A continuación, se presentan y discuten los resultados para cada categoría evaluada.

Los resultados completos de la auditoría Lighthouse para todas las páginas del sistema se presentan en el Anexo A.4.

1.5.1. Rendimiento

La categoría de rendimiento mide la velocidad de carga y la capacidad de respuesta de la página. Para ello, Lighthouse emplea las siguientes métricas:

- **Primer Despliegue de Contenido (FCP, por sus siglas en inglés):** tiempo que transcurre hasta que el navegador muestra el primer elemento visual de la página.
- **Despliegue del Contenido Más Grande (LCP):** tiempo necesario para renderizar el elemento de mayor tamaño visible para el usuario, siendo el principal indicador de velocidad de carga percibida.
- **Tiempo Total de Bloqueo (TBT):** tiempo durante el cual el hilo principal del navegador permanece bloqueado e incapaz de responder a las interacciones del usuario.
- **Índice de Velocidad (SI):** mide la rapidez con la que el contenido visible de la página se muestra progresivamente.
- **Desplazamiento Acumulado del Diseño (CLS):** cuantifica la estabilidad visual de la interfaz, reflejando los desplazamientos inesperados de los elementos durante la carga.

Las Tablas 1.4 y 1.5 muestran los resultados obtenidos para las páginas representativas del sistema.

Tabla 1.4: Resultados de rendimiento en entorno de escritorio.

Página	Puntaje	FCP	LCP	TBT	SI	CLS
Página principal	100	0.4 s	0.6 s	0 ms	0.7 s	0.00
Registro de ESAVI	100	0.4 s	0.6 s	0 ms	0.6 s	0.00
Panel informativo	100	0.4 s	0.6 s	0 ms	0.5 s	0.00

Tabla 1.5: Resultados de rendimiento en entorno móvil.

Página	Puntaje	FCP	LCP	TBT	SI	CLS
Página principal	90	2.2 s	2.9 s	0 ms	4.6 s	0.00
Registro de ESAVI	96	1.7 s	2.7 s	0 ms	1.7 s	0.00
Panel informativo	95	1.7 s	2.4 s	0 ms	3.9 s	0.00

Las tres páginas mostradas son representativas del sistema. Los resultados del resto de las páginas siguen la misma tendencia y se recogen en el Anexo A.4.

Los resultados en escritorio reflejan un rendimiento excepcional: las tres páginas alcanzan la puntuación máxima de 100, con métricas muy por debajo de los umbrales recomendados por Google (LCP menor a 2.5 s, CLS menor a 0.1)[1], lo que garantiza una experiencia instantánea y sin bloqueos.

En el entorno móvil, los puntajes se mantienen altos (90–96), aunque se observa un deterioro esperable por la simulación de red 4G lenta. La página principal es la más afectada, con un LCP de 2.9 s que supera ligeramente el umbral óptimo y un SI de 4.6 s que evidencia una carga progresiva más lenta. No obstante, el TBT nulo y el CLS perfecto se conservan en todas las páginas, lo que indica que la interfaz, una vez visible, responde sin demoras ni desplazamientos.

La auditoría de Lighthouse identificó los siguientes problemas que explican esta degradación en móvil:

- **Solicitudes que bloquean el renderizado:** la hoja de estilos principal (19.8 KiB) retrasa la renderización inicial en aproximadamente 300 ms. La solución pasaría por identificar y priorizar los estilos críticos necesarios para la primera visualización, difiriendo el resto.
- **JavaScript no utilizado:** el archivo principal (131.1 KiB) contiene 54.6 KiB de código prescindible en la carga inicial. La mejora consistiría en analizar qué porciones de código requiere cada página y separarlas para que se descarguen solo cuando sean necesarias.
- **Cadena de dependencias de red:** la solicitud al backend de autenticación introduce una latencia de ruta crítica de 515 ms. La solución requeriría investigar mecanismos para reducir el tiempo de establecimiento de conexión con dicho origen.

Estas oportunidades de mejora no se corrigieron durante el desarrollo de esta tesis porque no constituyen fallas del sistema, sino optimizaciones de configuración del entorno de despliegue. Dado que los puntajes obtenidos ya son satisfactorios, se decidió priorizar la implementación de las funcionalidades principales. Su aplicación en trabajos futuros permitiría reducir el LCP móvil por debajo de 2.5 s y llevar el rendimiento por encima de 95 puntos en todos los escenarios.

1.5.2. Accesibilidad

La categoría de accesibilidad evalúa qué tan usable es la página para personas con discapacidades, considerando aspectos como la compatibilidad con lectores de pantalla, la navegación por teclado y el contraste de colores.

La Tabla 1.6 muestra los puntajes obtenidos en cada página y entorno.

Tabla 1.6: Puntajes de accesibilidad por página y entorno.

Página	Escritorio	Móvil
Página principal	98	93
Registro de ESAVI	89	89
Panel informativo	98	95

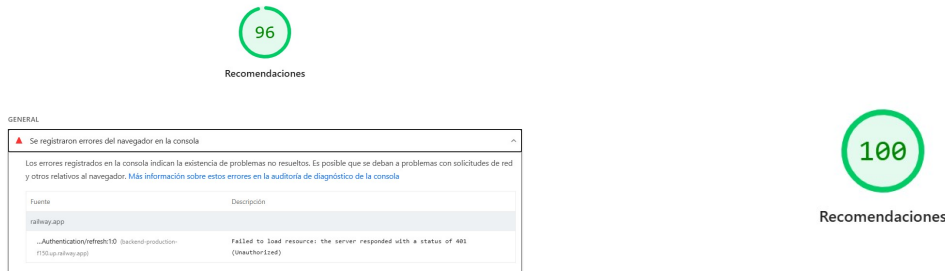
Los puntajes son altos en la mayoría de las páginas, aunque el Registro de ESAVI desciende a 89 en ambos entornos. La auditoría identificó tres problemas que explican esta diferencia:

- **Botones sin nombre accesible:** los desplegables de provincia y municipio carecen de un nombre descriptivo, por lo que un lector de pantalla los leerá simplemente como "botón". La solución pasaría por asignar atributos que permitan a las tecnologías de asistencia anunciar su función.
- **Barras de progreso sin etiqueta:** el indicador de avance del formulario no posee un nombre accesible, impidiendo que los lectores de pantalla informen su propósito. La mejora requeriría etiquetar este elemento para describir su función dentro del flujo de registro.
- **Jerarquía de encabezados incorrecta:** las etiquetas H1, H2, H3 no siguen un orden descendente secuencial, lo cual dificulta la navegación a usuarios con lectores de pantalla. La corrección requeriría revisar la estructura semántica de los componentes para garantizar una jerarquía lógica.

Estas incidencias afectan principalmente a usuarios que dependen de lectores de pantalla para interactuar con el sistema. Su resolución no requiere cambios funcionales profundos, sino una auditoría de los elementos interactivos y su etiquetado, lo que permitiría elevar el puntaje del Registro de ESAVI por encima de 90 y alcanzar valores cercanos a 100 en todas las páginas.

1.5.3. Mejores Prácticas

Esta categoría verifica que la página cumpla con los estándares de seguridad y desarrollo web moderno. El sistema obtuvo una puntuación uniforme de **96/100** en todas las páginas y entornos.



(a) Puntuación de 96 con el error de consola presente.

(b) Puntuación de 100 tras resolver la incidencia.

Figura 1.10: Incidencia de mejores prácticas y su corrección.

La única incidencia detectada fue un mensaje de error registrado en la consola del navegador durante la renovación del token de autenticación. El servidor respondía correctamente con un código 401 cuando no existía un token válido, pero la respuesta no era capturada antes de que se imprimiera en consola. Lighthouse interpreta cualquier registro en consola como una incidencia. El problema se resolvió manejando adecuadamente la respuesta 401 antes de que llegara a la consola, lo que elevó la puntuación a 100/100.

1.5.4. SEO

Esta categoría analiza qué tan optimizada está la página para los motores de búsqueda. El sistema obtuvo **91/100** de forma uniforme en todas las páginas y entornos. La auditoría detectó dos carencias puntuales:

- **Ausencia de meta descripción:** el documento HTML no incluye una etiqueta `<meta name="description">`, utilizada por los buscadores para generar el fragmento descriptivo en los resultados.
- **Archivo robots.txt no válido:** Vercel, al no encontrar un archivo `robots.txt` en el proyecto, devuelve el `index.html` en su lugar, lo que genera errores de sintaxis en la auditoría. La solución consistiría en agregar un archivo `robots.txt` básico que indique las reglas de rastreo.

Ambos problemas son superficiales y no afectan la operatividad del sistema. Con estas dos correcciones menores se alcanzaría un puntaje superior.

El sistema cumple satisfactoriamente con los estándares de calidad web en ambos entornos, con puntuaciones de excelencia en escritorio y valores aceptables en móvil. Las incidencias detectadas son superficiales y no comprometen la operatividad ni la experiencia del usuario.

1.6. Pruebas de usabilidad

La usabilidad del sistema se evaluó mediante un recorrido cognitivo con 7 participantes y el cuestionario SUS (System Usability Scale). Según Nielsen, 5 usuarios son suficientes para identificar aproximadamente el 85% de los problemas de usabilidad en una evaluación formativa [2], y se recomiendan de 3 a 4 participantes por grupo cuando existen perfiles diferenciados.

Dado que el sistema aún no se encuentra en producción, se trabajó con una muestra de conveniencia de dos perfiles: 5 ciudadanos (módulo público) y 2 administradores (gestión del sistema). No se consiguieron médicos ni jefes municipales para esta primera ronda. Las sesiones se realizaron de forma remota, lo que impidió registrar tiempos de ejecución y aplicar otras técnicas presenciales como el pensamiento en voz alta. Cada participante realizó tareas guiadas sin instrucciones previas y respondió el cuestionario SUS, compuesto por 10 afirmaciones con escala de Likert de 1 a 5.

1.6.1. Cuestionario SUS

1. Creo que me gustaría usar este sistema frecuentemente.
2. Encontré el sistema innecesariamente complejo.
3. Pensé que el sistema era fácil de usar.
4. Creo que necesitaría ayuda de un técnico para usar este sistema.
5. Encontré las funciones del sistema bien integradas.
6. Pensé que había demasiada inconsistencia en el sistema.
7. Imagino que la mayoría de las personas aprendería a usar este sistema rápidamente.
8. Encontré el sistema muy incómodo de usar.
9. Me sentí muy seguro al usar el sistema.
10. Necesité aprender muchas cosas antes de poder usar el sistema.

Para las afirmaciones impares (1, 3, 5, 7, 9), el puntaje es la valoración menos 1. Para las pares (2, 4, 6, 8, 10), es 5 menos la valoración. El resultado se multiplica por 2.5 para obtener una escala de 0 a 100.

$$\text{SUS} = 2,5 \times \left[\sum_{i=1,3,5,7,9} (v_i - 1) + \sum_{i=2,4,6,8,10} (5 - v_i) \right]$$

1.6.2. Resultados

La Tabla 1.7 presenta las valoraciones individuales. Las columnas P1 a P5 corresponden a ciudadanos y A1 a A2 a administradores.

Tabla 1.7: Valoraciones del cuestionario SUS por participante.

Afirmación	P1	P2	P3	P4	P5	A1	A2	Prom
1. Usaría el sistema frecuentemente	4	5	4	4	5	4	5	4.4
2. Sistema innecesariamente complejo	1	1	2	1	1	2	1	1.3
3. Sistema fácil de usar	5	4	4	5	4	4	5	4.4
4. Necesitaría ayuda técnica	1	2	1	1	2	1	1	1.3
5. Funciones bien integradas	4	5	4	4	5	4	5	4.4
6. Demasiada inconsistencia	1	1	2	1	1	2	1	1.3
7. Fácil de aprender	5	5	4	5	5	5	5	4.9
8. Sistema incómodo de usar	1	1	1	1	1	1	1	1.0
9. Me sentí seguro al usarlo	4	5	4	5	4	4	5	4.4
10. Necesité aprender antes	1	1	2	1	1	1	1	1.1
Puntaje SUS	92.5	90.0	80.0	92.5	90.0	82.5	95.0	89.0

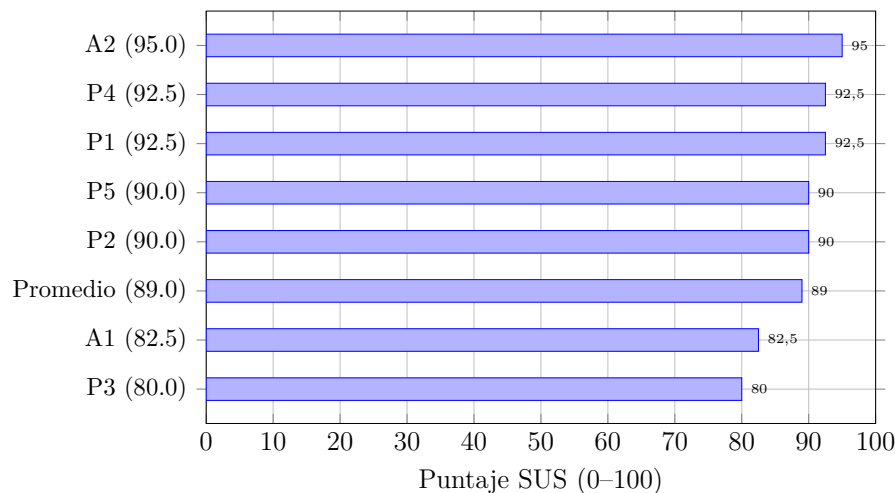


Figura 1.11: Puntaje SUS por participante, ordenado de menor a mayor.

1.6.3. Discusión

El puntaje SUS promedio de 89.0/100 sitúa al sistema en la categoría “excelente” según Bangor et al. [3], con todos los participantes por encima de 80. Sin embargo, este resultado solo es representativo de ciudadanos y administradores, y no puede extrapolarse a médicos ni jefes municipales sin pruebas adicionales. La modalidad remota impidió además registrar tiempos de ejecución y tasas de error. Se concluye que

la usabilidad es excelente para los dos perfiles evaluados, y se establecen a continuación los requisitos previos a la puesta en producción.

1.6.4. Requisitos previos a la puesta en producción

1. **Pruebas con médicos y jefes municipales:** aplicar el recorrido cognitivo y el cuestionario SUS a al menos 3 participantes de cada perfil, en sesiones presenciales o sincrónicas que permitan registrar tiempos de ejecución, tasa de errores y observaciones directas.
2. **Segunda iteración con rediseño:** incorporar los hallazgos, realizar los ajustes necesarios y reevaluar con otros 3 participantes por perfil, siguiendo el enfoque iterativo de Nielsen [2].

1.7. Discusión de resultados

Los resultados de las cuatro dimensiones de validación permiten una valoración integral del sistema.

En el plano funcional, todos los flujos principales operan correctamente conforme a los requerimientos, sin desviaciones en los casos de prueba ejecutados.

En el plano de rendimiento, el sistema mostró un comportamiento sólido bajo carga concurrente. La creación de reportes mantuvo una zona óptima hasta 70 VUs con el 100% de solicitudes por debajo de los 3000 ms, mientras que la búsqueda por notificación resultó la operación más rápida y consistente. El escenario combinado confirmó que los tres perfiles de uso conviven sin interferencia mutua, con el 100% de solicitudes exitosas. La degradación observada es progresiva y atribuible al entorno de pruebas, no a deficiencias del código.

En el plano de calidad web, el frontend alcanzó puntuaciones de 97 en rendimiento y 98 en accesibilidad en escritorio, con oportunidades de mejora en móvil que no comprometen la operatividad.

En el plano de usabilidad, el puntaje SUS de 89.0/100 sitúa al sistema en la categoría “excelente” para los perfiles evaluados, con una curva de aprendizaje baja.

En conjunto, el sistema cumple satisfactoriamente con los requerimientos definidos, ofreciendo una experiencia de usuario sólida tanto en condiciones normales como bajo carga concurrente.

Conclusiones

El desarrollo del sistema permitió materializar los objetivos planteados al inicio de este trabajo. Los principales logros alcanzados fueron:

- **Una arquitectura limpia en capas** que separa el dominio, la aplicación, la infraestructura y la presentación, y permitió migrar entre proveedores de correo sin modificar la lógica de negocio.
- **Una API REST segura** con autenticación basada en JWT, autorización por roles, CAPTCHA, rate limiting y validación en dos niveles.
- **Un sistema de notificaciones asíncrono** que desacopla el envío de correos mediante RabbitMQ, permitiendo que el usuario reciba respuesta inmediata.
- **Una interfaz de usuario responsiva** para que la población reporte eventos adversos y el personal del IFV gestione revisiones y asignaciones según su perfil.
- **La contenerización de la solución con Docker** y su despliegue en Railway y Vercel, con la misma configuración prevista para servidores físicos institucionales.

La experiencia abarcó desde el análisis de requisitos con expertos del dominio hasta la implementación de mecanismos de seguridad con fundamento criptográfico, pasando por la selección de tecnologías adecuadas al contexto cubano. El sistema resultante constituye una base funcional sobre la cual el Instituto Finlay puede construir las próximas iteraciones.

Recomendaciones

Más allá de los logros alcanzados, el proyecto deja abiertas varias líneas de evolución que permitirán al Instituto Finlay de Vacunas y a los desarrolladores que den continuidad al sistema llevarlo a su pleno potencial operativo. Se proponen las siguientes recomendaciones:

1. **Migración a servidores físicos del Instituto Finlay.** Se recomienda al equipo de infraestructura del IFV trasladar el sistema desde los entornos de desarrollo (Railway y Vercel) a los servidores institucionales, utilizando la misma configuración de contenedores Docker y variables de entorno ya desarrollada. Esta migración garantizará el control sobre los datos de farmacovigilancia y eliminará la dependencia de plataformas externas.
2. **Desarrollo de una aplicación móvil nativa.** Se recomienda al equipo de desarrollo evaluar la creación de una aplicación para iOS y Android que aproveche notificaciones push, acceso sin conexión a internet y una experiencia optimizada para pantallas táctiles. La API actual ya soporta esta evolución sin requerir modificaciones en el backend.
3. **Ampliación de los canales de notificación.** Se recomienda migrar progresivamente los servicios de notificación de prototipado a producción: un servidor SMTP institucional para el correo electrónico y la API oficial de WhatsApp Business para la mensajería. Adicionalmente, se sugiere incorporar SMS como canal complementario para alcanzar a usuarios sin acceso a datos móviles.
4. **Fusión inteligente de duplicados:** Actualmente, al confirmar un duplicado, el sistema descarta la copia incluso si contiene datos adicionales que el original no tiene. Una mejora futura consistiría en implementar una estrategia de fusión que, al confirmar el duplicado, preserve los datos únicos de la copia en el reporte original, siempre bajo supervisión del jefe de vacunación.

Estas recomendaciones se dirigen tanto al personal técnico del IFV encargado de la infraestructura y el desarrollo como a los directivos responsables de autorizar los contratos de servicios y la asignación de recursos necesarios para cada etapa.

Apéndice A

Código de las pruebas de rendimiento

A continuación se presentan los scripts de k6 utilizados en las pruebas de rendimiento del Capítulo 5, así como la configuración del entorno de pruebas.

A.1. Prueba de carga: creación de reporte ESAVI

A.1.1. Configuración del entorno de prueba

Cada solicitud de creación incluye: 13 vacunas del Instituto Finlay, 10 lotes por cada vacuna con número de lote, dosis, sitio de administración y fecha; datos del reportante y del paciente generados dinámicamente; y fechas de administración, inicio y fin del evento adverso predefinidas. Para evitar una base de datos completamente vacía, cada prueba se ejecutó con 1000 reportes precargados.

Cada prueba se configuró con etapas de rampa, carga estable y enfriamiento: 1 minuto de subida hasta el objetivo de VUs, 3 minutos de carga estable y 1 minuto de bajada a cero. Las pruebas fueron independientes para cada nivel de carga: 5, 10, 30, 50, 70, 80, 100, 150 y 200 VUs.

A.1.2. Script principal (load_test.js)

```
1 import http from 'k6/http';
2 import { check } from 'k6';
3 import { buildReport } from './payload.js';
4
5 const API_BASE = __ENV.API_BASE || 'http://192.168.XXX.XXX:5137/api'
6   ;
7 const TARGET = __ENV.TARGET || 100;
```

```

7
8 export const options = {
9   stages: [
10     { duration: '1m', target: Number(TARGET) },
11     { duration: '3m', target: Number(TARGET) },
12     { duration: '1m', target: 0 },
13   ],
14 };
15
16 export default function () {
17   const payload = buildReport(__VU, __ITER);
18
19   const res = http.post(
20     `${API_BASE}/Report/createPublic`,
21     JSON.stringify(payload),
22     { headers: { 'Content-Type': 'application/json' } }
23   );
24
25   check(res, {
26     'status 201': (r) => r.status === 201,
27     'tiempo < 3000ms': (r) => r.timings.duration < 3000,
28   });
29 }

```

A.1.3. Tabla de resultados completos

La Tabla A.1 presenta los valores detallados obtenidos en cada nivel de carga.

Tabla A.1: Resultados completos de la prueba de carga para creación de reportes.

VUs	Latencia (ms)	p(95) (ms)	Lat. sin red (ms)	Throug. (r/s)	<3000ms
5	106.6	177.4	[valor]	23.1	100%
10	188.6	327.7	[valor]	26.8	100%
30	304.1	627.9	[valor]	24.6	100%
50	546.2	1192.8	[valor]	23.0	100%
70	975.6	2214.1	[valor]	20.3	100%
80	1882.1	2956.2	[valor]	22.2	95.8%
100	2246.9	4056.4	[valor]	20.1	74.8%
150	4721.3	6728.2	[valor]	21.0	19.1%
200	5404.2	8011.1	[valor]	25.0	10.7%

A.2. Prueba de carga: búsqueda de reporte ESAVI

A.2.1. Configuración del Entorno para la Búsqueda

Las solicitudes de búsqueda seleccionan aleatoriamente un número de notificación de un archivo CSV con identificadores reales, generado para cada tamaño de base de datos (1k, 10k, 50k, 100k). Se empleó la misma configuración de etapas que en la prueba de creación. Los mecanismos de CAPTCHA y limitación por IP se desactivaron temporalmente, ya que las pruebas automatizadas no pueden resolverlos, lo que confirma que estas políticas de seguridad cumplen su función de frenar bots.

A.2.2. Script principal (search_test.js)

```
1 import http from 'k6/http';
2 import { check } from 'k6';
3 import papaparse from 'https://jslib.k6.io/papaparse/5.1.1/index.js'
4 ;
5 const CSV_FILE = __ENV.CSV_FILE || 'reports_1k.csv';
6 const csvData = papaparse.parse(open(`./${CSV_FILE}`), { header:
7   true }).data;
8 const BASE_URL = __ENV.API_BASE || 'http://192.168.XXX.XXX:5137/api'
9 ;
10 const TARGET = __ENV.TARGET || 10;
11
12 export const options = {
13   stages: [
14     { duration: '1m', target: Number(TARGET) },
15     { duration: '3m', target: Number(TARGET) },
16     { duration: '1m', target: 0 },
17   ],
18 };
19
20 export default function () {
21   const registro = csvData[Math.floor(Math.random() * csvData.
22     length)];
23   const notificationNumber = registro.notificationNumber;
24
25   const res = http.get(
26     `${BASE_URL}/Report/get-report?notificationNumber=${{
27     notificationNumber}&token=token-captcha`
28   );
29
30   check(res, {
31     'status 202': (r) => r.status === 202,
32     'tiempo < 3000ms': (r) => r.timings.duration < 3000,
```

```

29     });
30 }

```

A.2.3. Tabla de resultados completos

La Tabla A.2 presenta los valores detallados obtenidos en cada combinación de tamaño de base de datos y usuarios concurrentes.

Tabla A.2: Resultados completos de la prueba de carga para búsqueda de reportes.

VUs	BD 1k (ms)	BD 10k (ms)	BD 50k (ms)	BD 100k (ms)	<3000ms
5	[valor]	[valor]	[valor]	[valor]	[%]
10	[valor]	[valor]	[valor]	[valor]	[%]
30	[valor]	[valor]	[valor]	[valor]	[%]
50	[valor]	[valor]	[valor]	[valor]	[%]
100	[valor]	[valor]	[valor]	[valor]	[%]

A.3. Prueba de carga: escenario combinado

A.3.1. Configuración del entorno

El escenario combinado integra los tres perfiles de uso del sistema en una sola prueba: tráfico público (75 VUs, alternando 65% creación de reportes y 35% búsqueda por notificación, con pausas de 1 a 3 segundos), tráfico de jefes municipales (10 VUs, consultando el dashboard y asignando reportes a médicos, con pausas de 2 a 6 segundos) y tráfico de médicos (15 VUs, revisando su bandeja de asignaciones y evaluando reportes, con pausas de 1 a 4 segundos). Las pausas simulan el comportamiento humano real y evitan la saturación artificial. Las métricas se aislaron mediante tags para obtener resultados independientes por cada tipo de operación.

Los mecanismos de CAPTCHA y limitación por IP se desactivaron temporalmente por las mismas razones expuestas en la prueba de búsqueda.

A.3.2. Ejecución y procesamiento

La prueba se ejecutó con el siguiente comando, generando dos archivos de salida:

```
k6 run --out json=raw.json --summary-export=summary.json mixed2.js
```

El archivo `summary.json` contiene las métricas agregadas que el ecosistema de k6 calcula automáticamente: promedio, mediana, p(95), mínimo, máximo y número de solicitudes por cada tag definido. El archivo `raw.json` registra cada solicitud individual con su latencia y tag asociado. Este archivo crudo se procesó con un script de

Python para calcular las métricas adicionales requeridas por los diagramas de caja: cuartiles (Q1, Q3) y valores atípicos (outliers), definidos como aquellas solicitudes cuya latencia supera $Q3 + 1.5 \times (Q3 - Q1)$.

A.3.3. Script principal (mixed2.js)

```
1 import http from 'k6/http';
2 import { check, sleep } from 'k6';
3 import papaparse from './papaparse.js';
4 import { buildReport } from './create_report/base.js';
5
6 const BASE_URL = 'http://192.168.XXX.XXX:5137/api';
7 const csvData = papaparse.parse(open('./reports_1k.csv'), { header:
  true }).data;
8
9 const DASHBOARD_ENDPOINTS = [
10   '/MunicipalDashboard/overview',
11   '/MunicipalDashboard/doctors-performance',
12   '/MunicipalDashboard/stats_municipal',
13 ];
14
15 export const options = {
16   scenarios: {
17     trafico_publico: {
18       executor: 'constant-vus',
19       vus: 75,
20       duration: '1m',
21       exec: 'escenarioPublico',
22     },
23     trafico_jefes: {
24       executor: 'constant-vus',
25       vus: 10,
26       duration: '1m',
27       exec: 'escenarioJefes',
28     },
29     trafico_medicos: {
30       executor: 'constant-vus',
31       vus: 15,
32       duration: '1m',
33       exec: 'escenarioMedicos',
34     },
35   },
36   thresholds: {
37     'http_req_duration{name:CrearReporte}': ['p(95)<3000'],
38     'http_req_duration{name:BuscarReporte}': ['p(95)<3000'],
39     'http_req_duration{name:Dashboard_jefe}': ['p(95)<4000'],
40     // resto de thresholds por cada nombre relevante
```

```

41     },
42 };
43
44 export function escenarioPublico() {
45     const rand = Math.random();
46     if (rand < 0.65) {
47         const createReport = buildReport(__ITER);
48     } else {
49         const getReport = getReport(_ITER);
50     }
51     sleep(Math.random() * 2 + 1);
52 }
53
54 export function escenarioJefes() {
55     const rand = Math.random();
56
57     if (rand < 0.65) {
58         // Obtener reportes pendientes y evaluadores
59         const pendingRes = getReportsPendings(__ITER);
60
61         // asignar
62         const assignRes = assignReportToDoctor(__ITER);
63     } else {
64         const resDashboard = getDashboardData(__ITER);
65     }
66     sleep(Math.random() * 4 + 2);
67 }
68
69 export function escenarioMedicos() {
70
71     // buscar las asignaciones
72     const assignments = getMyAssignments(__ITER);
73
74     // evaluar
75     const evalRes = evaluateReport(__ITER, assignments);
76
77     sleep(Math.random() * 3 + 1);
78 }

```

A.3.4. Tabla de resultados completos

La Tabla A.3 presenta las métricas detalladas obtenidas para cada operación en el escenario combinado.

Tabla A.3: Resultados completos del escenario combinado con 100 VUs.

Métrica	Crear	Buscar	Pendientes	Médicos	Asignar	Asignaciones	Evaluar	Dashboard
Solicitudes	1078	581	84	84	84	365	68	38
Promedio (ms)	1021.37	268.04	423.15	275.19	603.64	348.86	433.61	428.00
Mediana (ms)	858.28	235.99	364.01	222.53	433.23	278.22	382.17	368.06
Q1 (ms)	616.87	157.37	223.25	153.75	291.09	180.81	276.81	208.54
Q3 (ms)	1170.59	324.28	484.20	358.44	626.59	427.35	515.97	480.45
p(95) (ms)	2468.39	551.48	1143.59	637.15	1805.32	783.83	911.90	1209.34
Mínimo (ms)	151.15	24.89	68.61	54.73	101.22	46.62	81.17	60.88
Máximo (ms)	4389.06	1389.79	1320.05	887.75	2571.61	1928.12	1262.24	1255.79
Outliers	80	25	8	4	12	18	4	4
% éxito	100%	100%	100%	100%	100%	100%	100%	100%

A.4. Resultados completos de la auditoría Lighthouse

La Tabla A.4 resume los puntajes por categoría para todas las páginas del sistema. En cada celda, el valor superior corresponde al entorno de escritorio y el inferior al móvil.

Tabla A.4: Puntajes generales de Lighthouse para todas las páginas del sistema.

Página	Rendimiento	Accesibilidad	Recomendaciones	SEO
Página principal (E)	100	98		
Página principal (M)	90	93	96	91
Registro de ESAVI (E)	100	89		
Registro de ESAVI (M)	96	89	96	91
Panel informativo (E)	100	98		
Panel informativo (M)	95	95	96	91
Consulta de reportes (E)	[]	[]		
Consulta de reportes (M)	[]	[]	96	91
Gestión de reportes (E)	[]	[]		
Gestión de reportes (M)	[]	[]	96	91
Gestión de médicos (E)	[]	[]		
Gestión de médicos (M)	[]	[]	96	91
Gestión de jefes (E)	[]	[]		
Gestión de jefes (M)	[]	[]	96	91
Gestión de catálogo (E)	100	94		
Gestión de catálogo (M)	95	94	100	91
Dashboard municipal (E)	[]	[]		
Dashboard municipal (M)	[]	[]	96	91
Inicio de sesión (E)	[]	[]		
Inicio de sesión (M)	[]	[]	96	91

(E) = Escritorio, (M) = Móvil. Mejores Prácticas y SEO son uniformes en todas las páginas.

La Tabla A.5 presenta las métricas de rendimiento detalladas para todas las páginas del sistema.

Tabla A.5: Métricas de rendimiento para todas las páginas del sistema.

Página	Puntaje	FCP	LCP	TBT	SI	CLS
	E / M	E / M	E / M	E / M	E / M	E / M
Página principal	100	0.4 s	0.6 s	0 ms	0.7 s	0.00
	90	2.2 s	2.9 s	0 ms	4.6 s	0.00
Gestión de catálogo	100	0.4 s	0.4 s	0 ms	0.6 s	0.029
	95	1.8 s	2.6 s	70 ms	2.9 s	0.00
En cada celda: valor superior = Escritorio (E), valor inferior = Móvil (M).						

Bibliografía

- [1] Google Developers. *Web Vitals*. Artículo oficial de Google sobre las métricas Web Vitals para medir la experiencia del usuario en la web. 2024. URL: <https://web.dev/articles/vitals> (vid. págs. 13, 15).
- [2] Jakob Nielsen. «Why You Only Need to Test with 5 Users». En: *Jakob Nielsen's Alertbox*. 2000. URL: <https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/> (vid. págs. 18, 20).
- [3] Aaron Bangor, Philip Kortum y James Miller. «Determining What Individual SUS Scores Mean: Adding an Adjective Rating Scale». En: *Journal of Usability Studies* 4.3 (2009), págs. 114-123 (vid. págs. 1, 19).